

Panduan Lengkap: Pengenaln Ucapan menggunakan VQ dan DTW

Kita akan membangun sistem yang dapat mengenali satu kalimat target: **"tiga tambah lima kurang dua sama dengan enam"**.

Sistem ini akan bekerja dalam dua fase utama:

1. **Fase Pelatihan (Training):** Membuat model referensi dari kalimat target.
2. **Fase Pengenaln (Recognition/Testing):** Membandingkan suara input baru dengan model referensi.

Blok Diagram Sistem

Cuplikan kode

```
graph TD
    %% Fase Pelatihan
    subgraph "FASE PELATIHAN (Membuat Referensi)"
        A1[Input Suara Referensi\n'tiga tambah lima kurang dua sama dengan enam'] --> B1{Desain Codebook VQ\n(e.g., K-Means)}
        B1 --> C1[Codebook (C)\n'Buku Kamus Vektor']
        C1 --> D1[Kuantisasi Vektor (VQ)]
        D1 --> E1[Sekuens Kuantisasi Referensi\n(Indeks Codebook) R]
    end

    %% Fase Pengenaln
    subgraph "FASE PENGENALAN (Pengujian)"
        A2[Input Suara Baru\ne.g., 'tiga tambah lima...'] --> B2{Ekstraksi Ciri\nMFCC}
        B2 --> C2[Kuantisasi Vektor\nMenggunakan Codebook C]
        C2 --> D2[Sekuens Kuantisasi Test\n(Indeks Codebook) T]

        D2 --> E2{Penghitungan Jarak\ne.g., Hamming/Euclidean}
        E2 --> F2[Optimasi Jalur DTW]
        F2 --> G2[Skor Jarak Akhir (D_min)]
        G2 --> H2{Ambang Batas (Threshold)}
        H2 -- "< Threshold" --> I2[Dikenali sebagai Kalimat Target]
        H2 -- "> Threshold" --> J2[Tidak Dikenali]
    end

    %% Gaya visual
    style A1 fill:#f9f,stroke:#333,stroke-width:2px
    style A2 fill:#ccf,stroke:#333,stroke-width:2px
    style D1 fill:#ffe082,stroke:#333,stroke-width:2px
    style H2 fill:#a5d6a7,stroke:#333,stroke-width:2px
```

```
style K1 fill:#81c784,stroke:#333,stroke-width:2px,stroke-dasharray: 5 5
```

```
style K2 fill:#e57373,stroke:#333,stroke-width:2px,stroke-dasharray: 5 5
```

Langkah 1: Akuisisi Data dan Pra-pemrosesan (Preprocessing)

Langkah awal adalah menangkap sinyal suara dan mengubahnya menjadi format yang dapat diproses komputer.

1.1 Rekaman Suara (Audio Recording)

Kita merekam pengucapan kalimat: "tiga tambah lima kurang dua sama dengan enam".

- **Parameter:** Misalkan disampling pada 16 kHz, 16-bit mono.
- **Sinyal Waktu:** Hasilnya adalah deretan angka (amplitudo) terhadap waktu.

1.2 Segmentasi dan Windowing

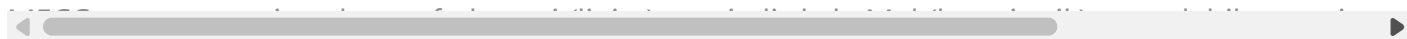
Sinyal suara tidak stasioner (berubah cepat). Untuk menganalisisnya, kita memotong sinyal menjadi frame-frame kecil yang tumpang tindih (overlapping).

- **Tujuan:** Agar sinyal di dalam setiap frame dianggap stasioner.
- **Ukuran Frame:** Umumnya 20-30 ms (e.g., 256 atau 512 sampel).
- **Overlap:** Umumnya 50% (e.g., 128 atau 256 sampel).
- **Window Function (misal: Hamming Window):** Dikalikan pada setiap frame untuk mengurangi kebocoran spektral (spectral leakage) di pinggir frame.

Langkah 2: Ekstraksi Ciri (Feature Extraction)

Tujuannya adalah mengubah setiap frame sinyal waktu menjadi sebuah vektor angka yang merepresentasikan karakteristik suara tersebut. Teknik standar yang digunakan adalah **Mel-Frequency Cepstral Coefficients (MFCC)**.

Rumus Utama MFCC:



dengan pendengaran manusia.

1. **FFT (Fast Fourier Transform):** Mengubah frame sinyal waktu $s(n)$ menjadi domain frekuensi $S(f)$.
2. **Mel Filterbank:** Menghitung energi dalam pita frekuensi skala Mel.
3. **Logarithm:** Mengambil logaritma dari energi tersebut: $L(m) = \log(\text{Energy}_m)$.
4. **DCT (Discrete Cosine Transform):** Mengubah log-energi kembali ke domain "cepstral".

$$C(n) = \frac{2}{M} \sum_{m=1}^M L(m) \cdot \cos\left(\frac{\pi n(m-0.5)}{M}\right)$$

Dimana:

- $C(n)$ adalah koefisien MFCC ke- n .
- M adalah jumlah filter dalam filterbank Mel.
- $L(m)$ adalah log-energi dari filter ke- m .

Visualisasi Output:

Sinyal suara kita sekarang direpresentasikan sebagai matriks fitur:

$$X = \{x_1, x_2, \dots, x_N\}$$

Dimana x_i adalah vektor MFCC (misal: 13 dimensi) untuk frame ke- i , dan N adalah jumlah total frame.

Langkah 3: Vector Quantization (VQ)

Ini adalah langkah **kompresi**. Kita memiliki ribuan vektor MFCC berdimensi tinggi. VQ mengelompokkan vektor-vektor serupa dan menggantinya dengan satu "kode" dari "codebook".

3.1 Desain Codebook (Training Phase)

Kita mengumpulkan semua vektor MFCC (X) dari data pelatihan dan menggunakan algoritma clustering, seperti **K-Means**, untuk membuat codebook.

Algoritma K-Means:

1. **Inisialisasi:** Pilih K centroid secara acak sebagai codeword awal $\{c_1, c_2, \dots, c_K\}$.
2. **Assignment:** Kelompokkan setiap vektor ciri x_i ke centroid terdekat.

$$\text{Assignment: } x_i \in \text{Cluster } k \quad \text{jika } k = \arg \min_{j=1 \dots K} \|x_i - c_j\|^2$$

3. **Update:** Hitung ulang setiap centroid sebagai rata-rata dari semua vektor dalam clusternya.

$$c_k = \frac{1}{|Cluster_k|} \sum_{x \in Cluster_k} x$$

4. **Repeat:** Ulangi langkah 2 dan 3 sampai centroid konvergen (tidak berubah signifikan).

Hasil: Codebook $C = \{c_1, c_2, \dots, c_K\}$, di mana K adalah ukuran codebook (misal: 64 atau 128). Setiap c_k adalah vektor 13 dimensi.

3.2 Kuantisasi Vektor (Fase Pelatihan & Pengenalan)

Setiap vektor ciri x_i (baik dari data referensi maupun data uji) digantikan oleh **indeks** (k) dari codeword terdekat dalam codebook C .

$$q(x_i) = \arg \min_{k=1 \dots K} \|x_i - c_k\|^2$$

Dimana $q(x_i)$ adalah fungsi kuantisasi yang mengembalikan indeks k .

3.3 Representasi Akhir

Suara referensi kita sekarang menjadi sekuens simbol diskrit (indeks codebook):

$$R = \{r_1, r_2, \dots, r_{N_R}\} = \{k_1, k_2, \dots, k_{N_R}\}$$

r_i adalah indeks codebook untuk frame ke- i dari sinyal referensi.

Suara input baru (test) juga diubah menjadi sekuens:

$$T = \{t_1, t_2, \dots, t_{N_T}\} = \{j_1, j_2, \dots, j_{N_T}\}$$

Dimana N_R dan N_T adalah jumlah frame (panjang sekuens), yang mungkin berbeda.

Visualisasi VQ

Cuplikan kode

```
graph TD
    %% Dataset Vektor
    subgraph "Kumpulan Vektor MFCC (Ruang Ciri)"
        V1((x1)) --- V2((x2))
        V2 --- V3((x3))
        V3 --- V4((x4))
        V1 --- V5((x5))
        %% DLL banyak vektor
        V5 -. -> VK((... xn))
    end

    %% Proses K-Means
    subgraph "Clustering K-Means"
        KM[Algoritma\K-Means]
    end

    V1 --> KM
    VK --> KM

    %% Hasil Codebook
    subgraph "Codebook (K Codewords)"
```

```

C1[Codeword 1\n(e.g., v1')]
C2[Codeword 2\n(e.g., v2')]
CK[Codeword K\n(e.g., vk')]

end

KM --> C1
KM --> C2
KM --> CK

%% Proses Kuantisasi
subgraph "Kuantisasi (Pengekodean)"
    InputX((xi)) --> |"Hitung Jarak\nterdekat"| Compare
    Compare --> C1
    Compare --> C2
    Compare --> CK
    Compare --> OutIndex[Index k\n(Simbol)]
end

```

Langkah 4: Dynamic Time Warping (DTW)

Ini adalah langkah **pembandingan pola**. Kita memiliki dua sekuens indeks codebook: R (referensi, "tiga...enam") dan T (test, misal "tiga...lima"). Kita perlu menghitung seberapa mirip kedua sekuens ini, meskipun diucapkan dengan kecepatan yang berbeda (durasi total berbeda).

DTW menemukan jalur warping optimal yang meminimalkan total jarak antara kedua sekuens.

4.1 Definisi Jarak Lokal

Pertama, kita butuh cara untuk mengukur jarak antara dua simbol (indeks codebook) r_i dan t_j . Ada dua pendekatan umum:

1. **Jarak Hamming (Simbolis):** Sangat sederhana. Jarak adalah 0 jika simbol sama, 1 jika berbeda.

$$d(r_i, t_j) = \begin{cases} 0 & \text{jika } r_i = t_j \\ 1 & \text{jika } r_i \neq t_j \end{cases}$$

2. **Jarak Euclidean (Vektor-Simbolis):** Lebih akurat. Jarak adalah jarak Euclidean antara *codeword* yang diwakili oleh indeks tersebut.

$$d(r_i, t_j) = \|c_{r_i} - c_{t_j}\|^2$$

Dimana c_{r_i} adalah vektor codeword ke- r_i dari codebook. Kita akan menggunakan ini untuk akurasi lebih baik.

4.2 Matriks Jarak Akumulasi (Accumulated Cost Matrix)

Kita membuat matriks D berukuran $N_R \times N_T$. Setiap sel $D(i, j)$ menyimpan biaya minimum untuk mencapai titik (i, j) dari titik awal $(1, 1)$.

Rumus Rekursif DTW:

$$D(i, j) = d(r_i, t_j) + \min \begin{cases} D(i-1, j) & \text{(Warping horizontal / Test diam)} \\ D(i, j-1) & \text{(Warping vertikal / Referensi diam)} \\ D(i-1, j-1) & \text{(Diagonal / Maju bersama)} \end{cases}$$

Kondisi Batas:

- $D(1, 1) = d(r_1, t_1)$
- $D(i, 1) = D(i-1, 1) + d(r_i, t_1)$ (untuk baris pertama)
- $D(1, j) = D(1, j-1) + d(r_1, t_j)$ (untuk kolom pertama)

4.3 Jarak DTW Akhir

Jarak DTW total adalah nilai akumulasi di sel terakhir matriks:

$$DTW(R, T) = D(N_R, N_T)$$

Biasanya, jarak ini dinormalisasi dengan total panjang jalur warping (L) untuk perbandingan yang adil:

$$\text{Normalisasi } DTW(R, T) = \frac{D(N_R, N_T)}{L}$$

Langkah 5: Visualisasi DTW dan Jalur Warping

Mari kita visualisasikan proses DTW. Asumsikan kita menggunakan jarak Euclidean antara codeword.

Visualisasi Jalur Warping:

Cuplikan kode

```
graph TD
    %% Sumbu Y: Sekuens Referensi R
    R_Start((R1)) --- R2((R2))
    R2 --- R3((R3))
    R3 -. -> RN((RN_R))

    subgraph "Sekuens Referensi (tiga ... enam)"
    end

    %% Sumbu X: Sekuens Test T
```

```

T_Start((T1)) --- T2((T2))
T2 --- T3((T3))
T3 -. -> TN((TN_T))

subgraph "Sekuens Test (tiga ... lima)"
end

%% Matriks Jarak Akumulasi (Konsep)
subgraph "Matriks Jarak DTW"
    M11[D(1,1)] --- M12[D(1,2)] --- M13[...] --- M1N[D(1,N_T)]
    M21[D(2,1)] --- M22[D(2,2)] --- M23[...] --- M2N[D(2,N_T)]
    M31[...] --- M32[...] --- M33[...] --- M3N[...]
    MN1[D(N_R,1)] --- MN2[D(N_R,2)] --- MN3[...] --- MNN[D(N_R,N_T)]
end

%% Jalur Warping Optimal (Contoh)
Path1((1,1)) --> Path2((2,1))
Path2 --> Path3((3,2))
Path3 --> Path4((4,3))
Path4 --> Path5((4,4))
Path5 --> PathLast((N_R,N_T))

%% Gaya visual jalur
style Path1 fill:#ff8a65,stroke:#333
style Path2 fill:#ff8a65,stroke:#333
style Path3 fill:#ff8a65,stroke:#333
style Path4 fill:#ff8a65,stroke:#333
style Path5 fill:#ff8a65,stroke:#333
style PathLast fill:#ff8a65,stroke:#333

```

Penjelasan Visualisasi:

- **Titik** (i, j) pada jalur menunjukkan bahwa frame i dari referensi dipasangkan dengan frame j dari input test.
- **Warping**: Jika jalur bergerak horisontal atau vertikal (e.g., $(2, 1) \rightarrow (3, 1)$), itu berarti satu frame di satu sekuens dipasangkan dengan beberapa frame di sekuens lainnya, mengkompensasi perbedaan kecepatan bicara.
- **Jalur Tebal**: Mewakili jalur optimal yang memberikan total biaya minimum $D(N_R, N_T)$.

6. Fase Pengenalan (Recognition)

1. Input suara baru (test) datang.
2. Lakukan Ekstraksi Ciri (MFCC).
3. Lakukan Vector Quantization (menggunakan codebook yang sama dari fase pelatihan) untuk mendapatkan sekuens indeks T .

4. Hitung jarak DTW antara T dan sekuens referensi R ("tiga tambah lima kurang dua sama dengan enam").
 5. **Keputusan:**
 - Jika $\text{DTW}(R, T) < \text{Threshold}$ (Ambang Batas), sistem memutuskan: "Kalimat Dikenali sebagai target".
 - Jika $\text{DTW}(R, T) > \text{Threshold}$, sistem memutuskan: "Bukan Kalimat Target (atau tidak dikenali)".
-

Kesimpulan dan Summary Langkah demi Langkah

Berikut ringkasan lengkap perancangannya:

1. **Training:**
 - Rekam: "tiga tambah lima kurang dua sama dengan enam".
 - Ciri: Ekstrak MFCC (vektor 13D per frame).
 - KMeans: Buat Codebook (misal 64 vektor 13D).
 - VQ: Kuantisasi MFCC referensi ke sekuens indeks $R = \{r_1, r_2, \dots\}$.
2. **Recognition (Test):**
 - Rekam suara input baru.
 - Ciri: Ekstrak MFCC.
 - VQ: Kuantisasi MFCC test ke sekuens indeks $T = \{t_1, t_2, \dots\}$.
3. **DTW:**
 - Buat Matriks D .
 - Gunakan rumus rekursif $D(i, j)$ dengan jarak Euclidean antara codeword.
 - Ambil jarak final $D(N_R, N_T)$.
4. **Decision:** Bandingkan jarak final dengan Threshold.

Ini adalah pendekatan klasik yang sangat baik untuk memahami fundamental pengenalan ucapan berbasis pola sebelum beralih ke model berbasis statistik seperti HMM (Hidden Markov Models) atau Deep Learning yang lebih modern.